

Hardhat + Ethers v6 智能合约部署完整教程

目录

1. [环境准备](#)
2. [Ethers v5 vs v6 主要变化](#)
3. [方式一：使用脚本部署](#)
4. [方式二：使用 Hardhat Deploy 插件](#)
5. [方式三：单元测试中部署](#)
6. [常见错误与解决方案](#)
7. [最佳实践](#)

环境准备

1. 初始化项目

```
mkdir my-dapp
cd my-dapp
npm init -y
```

2. 安装依赖

```
# 安装 Hardhat
npm install --save-dev hardhat

# 安装 Ethers v6 和 Hardhat 插件
npm install --save-dev @nomicfoundation/hardhat-ethers ethers

# 可选：安装其他有用的插件
npm install --save-dev @nomicfoundation/hardhat-verify
npm install --save-dev hardhat-gas-reporter
npm install --save-dev @nomicfoundation/hardhat-chai-matchers chai
```

注意：确保安装的是 `ethers` v6 版本 (6.x.x)

3. 初始化 Hardhat

```
npx hardhat init
```

选择 "Create a JavaScript project" 或 "Create a TypeScript project"

4. 配置 `hardhat.config.js`

```
require("@nomicfoundation/hardhat-ethers");
require("@nomicfoundation/hardhat-verify");
```

```

require("hardhat-gas-reporter");

module.exports = {
  solidity: {
    version: "0.8.20",
    settings: {
      optimizer: {
        enabled: true,
        runs: 200
      }
    }
  },
  networks: {
    // 本地网络
    hardhat: {
      chainId: 31337
    },
    localhost: {
      url: "http://127.0.0.1:8545"
    },
    // 测试网络
    sepolia: {
      url: process.env.SEPOLIA_RPC_URL || "",
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 11155111
    },
    goerli: {
      url: process.env.GOERLI_RPC_URL || "",
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 5
    },
    // 主网
    mainnet: {
      url: process.env.MAINNET_RPC_URL || "",
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 1
    }
  },
  etherscan: {
    apiKey: process.env.ETHERSCAN_API_KEY
  },
  gasReporter: {
    enabled: process.env.REPORT_GAS === "true",
    currency: "USD",
    coinmarketcap: process.env.COINMARKETCAP_API_KEY
  }
};

```

5. 创建 .env 文件

```
# .env
PRIVATE_KEY=your_private_key_here
SEPOLIA_RPC_URL=https://eth-sepolia.g.alchemy.com/v2/YOUR_API_KEY
GOERLI_RPC_URL=https://eth-goerli.g.alchemy.com/v2/YOUR_API_KEY
MAINNET_RPC_URL=https://eth-mainnet.g.alchemy.com/v2/YOUR_API_KEY
ETHERSCAN_API_KEY=your_ETHERSCAN_API_KEY
COINMARKETCAP_API_KEY=your_COINMARKETCAP_API_KEY
REPORT_GAS=false
```

安装 dotenv:

```
npm install --save-dev dotenv
```

在 `hardhat.config.js` 顶部添加:

```
require("dotenv").config();
```

Ethers v5 vs v6 主要变化

核心 API 变化对照表

功能	Ethers v5	Ethers v6
合约部署		
部署合约	<code>factory.deploy()</code>	<code>factory.deploy()</code> 或 <code>ethers.deployContract()</code>
等待部署完成	<code>contract.deployed()</code>	<code>contract.waitForDeployment()</code>
部署交易	<code>contract.deployTransaction</code>	<code>contract.deploymentTransaction</code>
合约地址		
获取地址	<code>contract.address</code>	<code>await contract.getAddress()</code> 或 <code>contract.target</code>
交易		
等待交易	<code>tx.wait()</code>	<code>tx.wait()</code> (相同)
交易哈希	<code>tx.hash</code>	<code>tx.hash</code> (相同)
Provider		
JSON-RPC Provider	<code>ethers.providers.JsonRpcProvider</code>	<code>ethers.JsonRpcProvider</code>
Web3 Provider	<code>ethers.providers.Web3Provider</code>	<code>ethers.BrowserProvider</code>
获取 Signer	<code>provider.getSigner()</code>	<code>await provider.getSigner()</code> (异步)
钱包		
创建钱包	<code>new ethers.Wallet(key, provider)</code>	<code>new ethers.Wallet(key, provider)</code> (相同)
工具函数		
解析以太	<code>ethers.utils.parseEther()</code>	<code>ethers.parseEther()</code>
格式化以太	<code>ethers.utils.formatEther()</code>	<code>ethers.formatEther()</code>
解析单位	<code>ethers.utils.parseUnits()</code>	<code>ethers.parseUnits()</code>
常量		
零地址	<code>ethers.constants.AddressZero</code>	<code>ethers.ZeroAddress</code>
最大 uint256	<code>ethers.constants.MaxUint256</code>	<code>ethers.MaxUint256</code>
BigNumber		
大数运算	<code>ethers.BigNumber.from()</code>	<code>BigInt()</code> (原生)
算术操作	<code>.add()</code> , <code>.sub()</code> , <code>.mul()</code> , <code>.div()</code>	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> (原生运算符)

重点变化说明

1. 部署方式变化

v5:

```
const contract = await factory.deploy(arg1, arg2);
await contract.deployed();
console.log("Address:", contract.address);
```

v6:

```
const contract = await factory.deploy(arg1, arg2);
await contract.waitForDeployment();
console.log("Address:", await contract.getAddress());
// 或使用 target 属性 (同步)
console.log("Address:", contract.target);
```

2. 地址获取变化

v5:

```
const address = contract.address; // 同步
```

v6:

```
const address = await contract.getAddress(); // 异步
// 或
const address = contract.target; // 同步
```

3. Provider 变化

v5:

```
const provider = new ethers.providers.JsonRpcProvider(url);
const signer = provider.getSigner(); // 同步
```

v6:

```
const provider = new ethers.JsonRpcProvider(url);
const signer = await provider.getSigner(); // 异步
```

4. Utils 变化

v5:

```
const amount = ethers.utils.parseEther("1.0");
const formatted = ethers.utils.formatEther(balance);
```

v6:

```
const amount = ethers.parseEther("1.0");
const formatted = ethers.formatEther(balance);
```

5. BigNumber 变化

v5:

```
const num1 = ethers.BigNumber.from("1000");
const num2 = ethers.BigNumber.from("500");
const result = num1.add(num2).mul(2);
```

v6:

```
const num1 = BigInt("1000");
const num2 = BigInt("500");
const result = (num1 + num2) * 2n; // 使用原生 BigInt
```

方式一：使用脚本部署

1. 创建合约文件

创建 `contracts/MyToken.sol`：

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MyToken {
    string public name;
    string public symbol;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;

    event Transfer(address indexed from, address indexed to, uint256 value);

    constructor(string memory _name, string memory _symbol, uint256 _initialSupply) {
        name = _name;
        symbol = _symbol;
        totalSupply = _initialSupply;
        balanceOf[msg.sender] = _initialSupply;
    }

    function transfer(address to, uint256 amount) public returns (bool) {
        require(balanceOf[msg.sender] >= amount, "Insufficient balance");
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        emit Transfer(msg.sender, to, amount);
        return true;
    }
}
```

2. 基础部署脚本

创建 `scripts/deploy.js`：

```
const hre = require("hardhat");
```

```

async function main() {
  console.log("开始部署 MyToken 合约...");

  // 1. 获取合约工厂
  const MyToken = await hre.ethers.getContractFactory("MyToken");

  // 2. 部署合约 (传入构造函数参数)
  const myToken = await MyToken.deploy(
    "My Token",           // name
    "MTK",                // symbol
    hre.ethers.parseEther("1000000") // initialSupply: 1,000,000 tokens
  );

  // 3. 等待部署完成
  await myToken.waitForDeployment();

  // 4. 获取合约地址
  const address = await myToken.getAddress();
  // 或使用 target 属性 (同步)
  // const address = myToken.target;

  console.log("MyToken 部署成功! ");
  console.log("合约地址:", address);

  // 5. 获取部署者地址
  const [deployer] = await hre.ethers.getSigners();
  console.log("部署者地址:", deployer.address);

  // 6. 验证合约状态
  const tokenName = await myToken.name();
  const tokenSymbol = await myToken.symbol();
  const supply = await myToken.totalSupply();

  console.log("\n合约信息:");
  console.log("- 名称:", tokenName);
  console.log("- 符号:", tokenSymbol);
  console.log("- 总供应量:", hre.ethers.formatEther(supply));

  // 7. 返回部署的合约实例 (可选)
  return myToken;
}

// 执行部署并处理错误
main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });

```

3. 高级部署脚本（带验证和等待）

创建 `scripts/deploy-advanced.js`：

```
const hre = require("hardhat");

async function main() {
  // 获取部署者账户
  const [deployer] = await hre.ethers.getSigners();
  console.log("使用账户部署:", deployer.address);

  // 查询账户余额
  const balance = await hre.ethers.provider.getBalance(deployer.address);
  console.log("账户余额:", hre.ethers.formatEther(balance), "ETH");

  // 部署参数
  const TOKEN_NAME = "My Token";
  const TOKEN_SYMBOL = "MTK";
  const INITIAL_SUPPLY = hre.ethers.parseEther("1000000");

  console.log("\n部署参数:");
  console.log("- 名称:", TOKEN_NAME);
  console.log("- 符号:", TOKEN_SYMBOL);
  console.log("- 初始供应量:", hre.ethers.formatEther(INITIAL_SUPPLY));

  // 获取合约工厂
  const MyToken = await hre.ethers.getContractFactory("MyToken");

  console.log("\n正在部署合约...");

  // 部署合约
  const myToken = await MyToken.deploy(TOKEN_NAME, TOKEN_SYMBOL, INITIAL_SUPPLY);

  // 等待部署完成
  await myToken.waitForDeployment();

  const contractAddress = myToken.target;
  console.log("✓ 合约部署成功!");
  console.log("✓ 合约地址:", contractAddress);

  // 获取部署交易信息
  const deployTx = myToken.deploymentTransaction;
  if (deployTx) {
    console.log("✓ 部署交易哈希:", deployTx.hash);
    console.log("✓ 部署区块号:", deployTx.blockNumber);

    // 等待多个区块确认（在测试网或主网上）
    if (hre.network.name !== "hardhat" && hre.network.name !== "localhost") {
      console.log("\n等待区块确认...");
      const receipt = await deployTx.wait(5); // 等待5个区块
      console.log("✓ 已确认", receipt.confirmations, "个区块");
    }
  }
}
```



```

    }
}

// 验证合约（仅在支持的网络上）
if (hre.network.config.chainId === 11155111 && process.env.ETHERSCAN_API_KEY) {
  console.log("\n等待 Etherscan 索引...");
  await sleep(30000); // 等待30秒

  console.log("开始验证合约...");
  try {
    await hre.run("verify:verify", {
      address: contractAddress,
      constructorArguments: [TOKEN_NAME, TOKEN_SYMBOL, INITIAL_SUPPLY],
    });
    console.log("✓ 合约验证成功!");
  } catch (error) {
    console.log("✗ 合约验证失败:", error.message);
  }
}

// 保存部署信息
saveFrontendFiles(myToken);

return myToken;
}

// 辅助函数：延迟
function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

// 辅助函数：保存部署信息到前端
function saveFrontendFiles(contract) {
  const fs = require("fs");
  const contractsDir = __dirname + "/../frontend/contracts";

  if (!fs.existsSync(contractsDir)) {
    fs.mkdirSync(contractsDir, { recursive: true });
  }

  // 保存合约地址
  fs.writeFileSync(
    contractsDir + "/contract-address.json",
    JSON.stringify({ MyToken: contract.target }, undefined, 2)
  );

  // 保存合约 ABI
  const MyTokenArtifact = hre.artifacts.readArtifactSync("MyToken");
  fs.writeFileSync(
    contractsDir + "/MyToken.json",
    JSON.stringify(MyTokenArtifact, null, 2)
  );
};

```

```

    console.log("\n✓ 部署信息已保存到 frontend/contracts/");
  }

  main()
    .then(() => process.exit(0))
    .catch((error) => {
      console.error(error);
      process.exit(1);
    });
});

```

4. 运行部署脚本

```

# 部署到本地 Hardhat 网络
npx hardhat run scripts/deploy.js

# 部署到本地节点（需要先启动节点）
npx hardhat node # 在另一个终端
npx hardhat run scripts/deploy.js --network localhost

# 部署到 Sepolia 测试网
npx hardhat run scripts/deploy.js --network sepolia

# 部署到主网（谨慎！）
npx hardhat run scripts/deploy.js --network mainnet

```

5. 部署多个合约的脚本

创建 `scripts/deploy-all.js`：

```

const hre = require("hardhat");

async function main() {
  const [deployer] = await hre.ethers.getSigners();
  console.log("部署者地址:", deployer.address);

  const deployedContracts = {};

  // 1. 部署 Token 合约
  console.log("\n1. 部署 MyToken...");
  const MyToken = await hre.ethers.getContractFactory("MyToken");
  const myToken = await MyToken.deploy(
    "My Token",
    "MTK",
    hre.ethers.parseEther("1000000")
  );
  await myToken.waitForDeployment();
  deployedContracts.MyToken = myToken.target;
  console.log("✓ MyToken 地址:", myToken.target);
}

```

```

// 2. 部署 NFT 合约
console.log("\n2. 部署 MyNFT...");
const MyNFT = await hre.ethers.getContractFactory("MyNFT");
const myNFT = await MyNFT.deploy("My NFT", "MNFT");
await myNFT.waitForDeployment();
deployedContracts.MyNFT = myNFT.target;
console.log("✓ MyNFT 地址:", myNFT.target);

// 3. 部署依赖其他合约的合约
console.log("\n3. 部署 TokenSale...");
const TokenSale = await hre.ethers.getContractFactory("TokenSale");
const tokenSale = await TokenSale.deploy(
  myToken.target, // 使用已部署的 Token 地址
  hre.ethers.parseEther("0.001") // 价格
);
await tokenSale.waitForDeployment();
deployedContracts.TokenSale = tokenSale.target;
console.log("✓ TokenSale 地址:", tokenSale.target);

// 4. 执行初始化操作
console.log("\n4. 执行初始化...");

// 给 TokenSale 合约转账一些 Token
const transferTx = await myToken.transfer(
  tokenSale.target,
  hre.ethers.parseEther("100000")
);
await transferTx.wait();
console.log("✓ 已转账 100,000 MTK 到 TokenSale");

// 输出所有部署的合约地址
console.log("\n===== 部署完成 =====");
console.log(JSON.stringify(deployedContracts, null, 2));

// 保存到文件
const fs = require("fs");
fs.writeFileSync(
  "./deployed-contracts.json",
  JSON.stringify(deployedContracts, null, 2)
);
console.log("\n✓ 部署信息已保存到 deployed-contracts.json");
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });

```

6. 使用 ethers.deployContract() 快捷方式

```

async function main() {
  // 方式 A: 使用 ContractFactory
  const MyToken = await hre.ethers.getContractFactory("MyToken");
  const myToken = await MyToken.deploy("My Token", "MTK",
hre.ethers.parseEther("1000000"));
  await myToken.waitForDeployment();

  // 方式 B: 使用 deployContract 快捷方式 (v6 新增)
  const myToken2 = await hre.ethers.deployContract("MyToken", [
    "My Token",
    "MTK",
    hre.ethers.parseEther("1000000")
  ]);
  await myToken2.waitForDeployment();

  console.log("Token 1:", myToken.target);
  console.log("Token 2:", myToken2.target);
}

```

方式二：使用 Hardhat Deploy 插件

Hardhat Deploy 是一个强大的插件，提供了更好的部署管理、升级支持和多链部署。

1. 安装 Hardhat Deploy

```

npm install --save-dev hardhat-deploy
npm install --save-dev @nomicfoundation/hardhat-ethers ethers

```

2. 配置 hardhat.config.js

```

require("@nomicfoundation/hardhat-ethers");
require("hardhat-deploy");

module.exports = {
  solidity: "0.8.20",
  networks: {
    hardhat: {
      chainId: 31337
    },
    sepolia: {
      url: process.env.SEPOLIA_RPC_URL,
      accounts: [process.env.PRIVATE_KEY],
      chainId: 11155111
    }
  },
  namedAccounts: {
    deployer: {
      default: 0, // 默认使用第一个账户作为部署者
    }
  }
}

```

```

    1: 0,          // 主网也使用第一个账户
    11155111: 0 // Sepolia 使用第一个账户
  },
  user1: {
    default: 1 // 测试用户
  }
}
};

```

3. 创建部署脚本

创建 `deploy/01-deploy-token.js` :

```

module.exports = async ({ getNamedAccounts, deployments }) => {
  const { deploy, log } = deployments;
  const { deployer } = await getNamedAccounts();

  log("-----");
  log("部署 MyToken 合约...");

  const myToken = await deploy("MyToken", {
    from: deployer,
    args: [
      "My Token",
      "MTK",
      ethers.parseEther("1000000")
    ],
    log: true,
    waitConfirmations: network.config.blockConfirmations || 1,
  });

  log(`MyToken 部署到: ${myToken.address}`);
  log("-----");
};

module.exports.tags = ["all", "token"];

```

4. 创建多个部署文件

`deploy/02-deploy-nft.js` :

```

module.exports = async ({ getNamedAccounts, deployments }) => {
  const { deploy, log } = deployments;
  const { deployer } = await getNamedAccounts();

  log("部署 MyNFT 合约...");

  const myNFT = await deploy("MyNFT", {
    from: deployer,
    args: ["My NFT", "MNFT"],
  });

```

```

    log: true,
    waitConfirmations: 1,
  });

  log(`MyNFT 部署到: ${myNFT.address}`);
};

module.exports.tags = ["all", "nft"];

```

deploy/03-deploy-sale.js:

```

module.exports = async ({ getNamedAccounts, deployments, ethers }) => {
  const { deploy, log, get } = deployments;
  const { deployer } = await getNamedAccounts();

  // 获取之前部署的 Token 合约地址
  const myToken = await get("MyToken");

  log("部署 TokenSale 合约...");

  const tokenSale = await deploy("TokenSale", {
    from: deployer,
    args: [
      myToken.address,
      ethers.parseEther("0.001")
    ],
    log: true,
    waitConfirmations: 1,
  });

  log(`TokenSale 部署到: ${tokenSale.address}`);

  // 执行初始化
  if (network.name !== "hardhat") {
    log("执行初始化...");
    const token = await ethers.getContractAt("MyToken", myToken.address);
    const tx = await token.transfer(
      tokenSale.address,
      ethers.parseEther("100000")
    );
    await tx.wait();
    log("✓ 已转账 Token 到 TokenSale");
  }
};

module.exports.tags = ["all", "sale"];
module.exports.dependencies = ["token"]; // 依赖 token 部署

```

5. 运行 Hardhat Deploy

```

# 部署所有脚本
npx hardhat deploy

# 部署特定标签
npx hardhat deploy --tags token
npx hardhat deploy --tags nft,sale

# 部署到特定网络
npx hardhat deploy --network sepolia

# 重置并重新部署
npx hardhat deploy --reset

# 导出部署信息
npx hardhat deploy --export deployments.json

```

6. 在测试中使用已部署的合约

```

const { deployments, ethers } = require("hardhat");

describe("MyToken", function () {
  before(async function () {
    // 部署所有合约
    await deployments.fixture(["all"]);

    // 获取已部署的合约
    const myTokenDeployment = await deployments.get("MyToken");
    this.myToken = await ethers.getContractAt("MyToken", myTokenDeployment.address);

    // 获取签名者
    const signers = await ethers.getSigners();
    this.deployer = signers[0];
    this.user1 = signers[1];
  });

  it("应该有正确的初始供应量", async function () {
    const supply = await this.myToken.totalSupply();
    expect(supply).to.equal(ethers.parseEther("1000000"));
  });
});

```

7. Hardhat Deploy 高级功能

代理部署（可升级合约）

deploy/04-deploy-upgradeable.js :

```

module.exports = async ({ getNamedAccounts, deployments }) => {
  const { deploy } = deployments;
  const { deployer } = await getNamedAccounts();

```

```

await deploy("MyUpgradeableToken", {
  from: deployer,
  args: [],
  log: true,
  proxy: {
    proxyContract: "OpenZeppelinTransparentProxy",
    execute: {
      init: {
        methodName: "initialize",
        args: ["My Token", "MTK", ethers.parseEther("1000000")]
      }
    }
  }
});
};

module.exports.tags = ["upgradeable"];

```

条件部署

```

module.exports = async ({ getNamedAccounts, deployments, network }) => {
  const { deploy } = deployments;
  const { deployer } = await getNamedAccounts();

  // 仅在测试网部署
  if (network.name === "sepolia" || network.name === "goerli") {
    await deploy("TestToken", {
      from: deployer,
      args: ["Test Token", "TEST"],
      log: true,
    });
  }
};

```

方式三：单元测试中部署

1. 基础测试

创建 `test/MyToken.test.js`:

```

const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("MyToken 合约测试", function () {
  let myToken;
  let owner;
  let addr1;
  let addr2;

```



```

// 每个测试前都重新部署合约
beforeEach(async function () {
  // 获取测试账户
  [owner, addr1, addr2] = await ethers.getSigners();

  // 部署合约
  const MyToken = await ethers.getContractFactory("MyToken");
  myToken = await MyToken.deploy(
    "My Token",
    "MTK",
    ethers.parseEther("1000000")
  );
  await myToken.waitForDeployment();
});

describe("部署", function () {
  it("应该设置正确的名称和符号", async function () {
    expect(await myToken.name()).to.equal("My Token");
    expect(await myToken.symbol()).to.equal("MTK");
  });

  it("应该给部署者分配总供应量", async function () {
    const ownerBalance = await myToken.balanceOf(owner.address);
    expect(await myToken.totalSupply()).to.equal(ownerBalance);
  });
});

describe("交易", function () {
  it("应该在账户之间转账 Token", async function () {
    // 从 owner 转账 50 个 token 到 addr1
    await myToken.transfer(addr1.address, ethers.parseEther("50"));
    const addr1Balance = await myToken.balanceOf(addr1.address);
    expect(addr1Balance).to.equal(ethers.parseEther("50"));

    // 从 addr1 转账 50 个 token 到 addr2
    await myToken.connect(addr1).transfer(addr2.address, ethers.parseEther("50"));
    const addr2Balance = await myToken.balanceOf(addr2.address);
    expect(addr2Balance).to.equal(ethers.parseEther("50"));
  });

  it("应该失败如果发送者余额不足", async function () {
    const initialOwnerBalance = await myToken.balanceOf(owner.address);

    // 尝试从 addr1 (余额为 0) 转账 1 个 token 到 owner
    await expect(
      myToken.connect(addr1).transfer(owner.address, ethers.parseEther("1"))
    ).to.be.revertedWith("Insufficient balance");

    // Owner 余额不应该改变
    expect(await myToken.balanceOf(owner.address)).to.equal(initialOwnerBalance);
  });
});

```

```

it("应该更新余额", async function () {
  const initialOwnerBalance = await myToken.balanceOf(owner.address);

  // 转账 100 tokens 到 addr1
  await myToken.transfer(addr1.address, ethers.parseEther("100"));

  // 转账 50 tokens 到 addr2
  await myToken.transfer(addr2.address, ethers.parseEther("50"));

  // 检查余额
  const finalOwnerBalance = await myToken.balanceOf(owner.address);
  expect(finalOwnerBalance).to.equal(
    initialOwnerBalance - ethers.parseEther("150")
  );

  const addr1Balance = await myToken.balanceOf(addr1.address);
  expect(addr1Balance).to.equal(ethers.parseEther("100"));

  const addr2Balance = await myToken.balanceOf(addr2.address);
  expect(addr2Balance).to.equal(ethers.parseEther("50"));
});

describe("事件", function () {
  it("应该触发 Transfer 事件", async function () {
    await expect(myToken.transfer(addr1.address, ethers.parseEther("50")))
      .to.emit(myToken, "Transfer")
      .withArgs(owner.address, addr1.address, ethers.parseEther("50"));
  });
});
});

```

2. 使用 Fixture 优化测试

```

const { expect } = require("chai");
const { ethers } = require("hardhat");
const { loadFixture } = require("@nomicfoundation/hardhat-network-helpers");

describe("MyToken 合约测试 (使用 Fixture) ", function () {
  // 定义 fixture
  async function deployTokenFixture() {
    const [owner, addr1, addr2] = await ethers.getSigners();

    const MyToken = await ethers.getContractFactory("MyToken");
    const myToken = await MyToken.deploy(
      "My Token",
      "MTK",
      ethers.parseEther("1000000")
    );
    await myToken.waitForDeployment();
  }

```

```

// Fixtures 可以返回任何你需要的内容
return { myToken, owner, addr1, addr2 };
}

describe("部署", function () {
  it("应该设置正确的名称", async function () {
    const { myToken } = await loadFixture(deployTokenFixture);
    expect(await myToken.name()).to.equal("My Token");
  });

  it("应该分配总供应量给所有者", async function () {
    const { myToken, owner } = await loadFixture(deployTokenFixture);
    const ownerBalance = await myToken.balanceOf(owner.address);
    expect(await myToken.totalSupply()).to.equal(ownerBalance);
  });
});

describe("交易", function () {
  it("应该转账 Token", async function () {
    const { myToken, owner, addr1 } = await loadFixture(deployTokenFixture);

    await myToken.transfer(addr1.address, ethers.parseEther("50"));
    expect(await myToken.balanceOf(addr1.address)).to.equal(
      ethers.parseEther("50")
    );
  });
});
});
});

```

3. 测试 Gas 消耗

```

describe("Gas 优化测试", function () {
  it("应该报告交易的 Gas 消耗", async function () {
    const { myToken, addr1 } = await loadFixture(deployTokenFixture);

    const tx = await myToken.transfer(addr1.address, ethers.parseEther("100"));
    const receipt = await tx.wait();

    console.log("Gas 消耗:", receipt.gasUsed.toString());
    expect(receipt.gasUsed).to.be.lt(100000); // 应该少于 100k gas
  });
});

```

4. 时间旅行测试

```

const { time } = require("@nomicfoundation/hardhat-network-helpers");

describe("时间锁测试", function () {
  it("应该在时间锁期后允许提取", async function () {

```

```
const { timeLock, owner } = await loadFixture(deployTimeLockFixture);

// 存款
await timeLock.deposit({ value: ethers.parseEther("1") });

// 尝试立即提取 (应该失败)
await expect(timeLock.withdraw()).to.be.revertedWith("Time lock not expired");

// 快进时间 1 天
await time.increase(24 * 60 * 60);

// 现在应该可以提取
await expect(timeLock.withdraw()).to.not.be.reverted;
});
});
```

5. 运行测试

```
# 运行所有测试
npx hardhat test

# 运行特定测试文件
npx hardhat test test/MyToken.test.js

# 使用 grep 过滤测试
npx hardhat test --grep "应该转账"

# 显示 Gas 报告
REPORT_GAS=true npx hardhat test

# 显示详细输出
npx hardhat test --verbose

# 测试覆盖率
npx hardhat coverage
```

常见错误与解决方案

错误 1: "contract.deployed is not a function"

原因: 在 ethers v6 中, `deployed()` 已改为 `waitForDeployment()`

v5 代码:

```
await contract.deployed();
```

v6 修复:

```
await contract.waitForDeployment();
```

错误 2: "contract.deployTransaction is undefined"

原因: v6 中改为 `deploymentTransaction` (注意多了 "ment")

v5 代码:

```
const tx = contract.deployTransaction;  
await tx.wait(5);
```

v6 修复:

```
const tx = contract.deploymentTransaction;  
if (tx) {  
  await tx.wait(5);  
}
```

错误 3: "contract.address is undefined"

原因: v6 中地址获取方式改变

v5 代码:

```
console.log(contract.address);
```

v6 修复:

```
// 方式 1: 异步获取  
console.log(await contract.getAddress());  
  
// 方式 2: 同步属性  
console.log(contract.target);
```

错误 4: "ethers.utils is undefined"

原因: v6 中工具函数直接挂在 ethers 上

v5 代码:

```
ethers.utils.parseEther("1.0");  
ethers.utils.formatEther(balance);
```

v6 修复:

```
ethers.parseEther("1.0");  
ethers.formatEther(balance);
```

错误 5: "Cannot read property 'add' of undefined"

原因: v6 使用原生 BigInt 而非 BigNumber

v5 代码:

```
const result = num1.add(num2);
const doubled = result.mul(2);
```

v6 修复:

```
const result = num1 + num2;
const doubled = result * 2n; // 注意使用 BigInt 字面量
```

错误 6: "provider.getSigner is not a function"

原因: v6 中 getSigner 需要 await

v5 代码:

```
const signer = provider.getSigner();
```

v6 修复:

```
const signer = await provider.getSigner();
```

错误 7: "insufficient funds for intrinsic transaction cost"

原因: 账户余额不足

解决方案:

```
// 检查余额
const balance = await ethers.provider.getBalance(deployer.address);
console.log("余额:", ethers.formatEther(balance), "ETH");

// 在测试网获取测试币
// Sepolia: https://sepoliafaucet.com/
// Goerli: https://goerlifaucet.com/
```

错误 8: "nonce has already been used"

原因: Nonce 冲突

解决方案:

```
// 方案 1: 等待交易完成
const tx1 = await contract.someFunction();
await tx1.wait();

const tx2 = await contract.anotherFunction();
await tx2.wait();

// 方案 2: 手动指定 nonce
const nonce = await provider.getTransactionCount(deployer.address);
const tx = await contract.someFunction({ nonce: nonce });
```

错误 9: "Contract deployment failed"

调试步骤:

```
try {
  const contract = await factory.deploy(...args);
  await contract.waitForDeployment();
} catch (error) {
  console.error("部署失败!");
  console.error("错误消息:", error.message);
  console.error("错误代码:", error.code);
  console.error("完整错误:", error);

  // 检查构造函数参数
  console.log("构造函数参数:", args);

  // 检查 Gas 限制
  console.log("Gas 限制:", error.transaction?.gasLimit);
}
```

最佳实践

1. 环境管理

```
// hardhat.config.js
require("dotenv").config();

function getEnvOrThrow(key) {
  const value = process.env[key];
  if (!value) {
    throw new Error(`环境变量 ${key} 未设置`);
  }
  return value;
}

module.exports = {
  networks: {
```

```
sepolia: {
  url: getEnvOrThrow("SEPOLIA_RPC_URL"),
  accounts: [getEnvOrThrow("PRIVATE_KEY")],
}
}
};
```

2. 安全部署检查清单

```
async function safeDeployment() {
  // 1. 检查网络
  const network = await ethers.provider.getNetwork();
  console.log("当前网络:", network.name, "Chain ID:", network.chainId);

  if (network.chainId === 1) {
    console.log("警告: 即将部署到主网!");
    // 在生产中, 添加额外的确认步骤
  }

  // 2. 检查部署者余额
  const [deployer] = await ethers.getSigners();
  const balance = await ethers.provider.getBalance(deployer.address);
  console.log("部署者余额:", ethers.formatEther(balance), "ETH");

  if (balance < ethers.parseEther("0.1")) {
    throw new Error("余额不足以部署合约");
  }

  // 3. 估算部署 Gas
  const MyToken = await ethers.getContractFactory("MyToken");
  const deployTx = await MyToken.getDeployTransaction(
    "My Token",
    "MTK",
    ethers.parseEther("1000000")
  );
  const estimatedGas = await ethers.provider.estimateGas(deployTx);
  console.log("预估 Gas:", estimatedGas.toString());

  // 4. 部署
  const myToken = await MyToken.deploy(
    "My Token",
    "MTK",
    ethers.parseEther("1000000")
  );
  await myToken.waitForDeployment();

  // 5. 验证部署
  const code = await ethers.provider.getCode(myToken.target);
  if (code === "0x") {
    throw new Error("合约部署失败: 没有字节码");
  }
}
```



```

    console.log("✓ 部署成功:", myToken.target);
    return myToken;
}

```

3. 部署信息记录

```

const fs = require("fs");
const path = require("path");

function saveDeployment(contractName, address, args, network) {
    const deploymentsDir = path.join(__dirname, "../deployments");
    if (!fs.existsSync(deploymentsDir)) {
        fs.mkdirSync(deploymentsDir, { recursive: true });
    }

    const deployment = {
        contractName,
        address,
        constructorArgs: args,
        network: network.name,
        chainId: network.chainId,
        deployedAt: new Date().toISOString(),
        deployer: deployer.address
    };

    const filename = path.join(
        deploymentsDir,
        `${contractName}-${network.name}.json`
    );

    fs.writeFileSync(filename, JSON.stringify(deployment, null, 2));
    console.log(`✓ 部署信息已保存到 ${filename}`);
}

// 使用
await saveDeployment(
    "MyToken",
    myToken.target,
    ["My Token", "MTK", ethers.parseEther("1000000")],
    await ethers.provider.getNetwork()
);

```

4. 多签部署

```

async function multiSigDeploy() {
    // 1. 准备部署交易数据
    const MyToken = await ethers.getContractFactory("MyToken");
    const deployTx = MyToken.getDeployTransaction(
        "My Token",

```

```

    "MTK",
    ethers.parseEther("1000000")
  );

// 2. 编码交易数据
const txData = {
  to: null, // 部署交易
  value: 0,
  data: deployTx.data
};

console.log("请在多签钱包中提交以下交易:");
console.log(JSON.stringify(txData, null, 2));

// 3. 等待多签执行...
// 在实际应用中, 你需要监听多签钱包的事件
}

```

5. 升级准备

```

// 部署可升级合约的准备
async function deployUpgradeable() {
  const { ethers, upgrades } = require("hardhat");

  const MyTokenV1 = await ethers.getContractFactory("MyTokenV1");

  // 使用 OpenZeppelin Upgrades 插件
  const proxy = await upgrades.deployProxy(MyTokenV1, [
    "My Token",
    "MTK",
    ethers.parseEther("1000000")
  ], {
    initializer: "initialize",
    kind: "uups" // 或 "transparent"
  });

  await proxy.waitForDeployment();
  console.log("代理地址:", proxy.target);
  console.log("实现地址:", await upgrades.erc1967.getImplementationAddress(proxy.target));
}

```

6. 使用配置文件管理部署参数

创建 `config/deployment.json`:

```

{
  "hardhat": {
    "MyToken": {
      "name": "My Token (Local)",
      "symbol": "MTK",

```

```

    "initialSupply": "1000000"
  }
},
"sepolia": {
  "MyToken": {
    "name": "My Token (Sepolia)",
    "symbol": "MTK",
    "initialSupply": "1000000"
  }
},
"mainnet": {
  "MyToken": {
    "name": "My Token",
    "symbol": "MTK",
    "initialSupply": "10000000"
  }
}
}
}

```

使用配置：

```

const config = require("../config/deployment.json");

async function main() {
  const network = await ethers.provider.getNetwork();
  const networkName = network.name;
  const params = config[networkName].MyToken;

  const MyToken = await ethers.getContractFactory("MyToken");
  const myToken = await MyToken.deploy(
    params.name,
    params.symbol,
    ethers.parseEther(params.initialSupply)
  );

  await myToken.waitForDeployment();
  console.log("MyToken 部署到:", myToken.target);
}

```

总结

ethers v6 部署核心要点

1. 部署合约

```

const contract = await factory.deploy(...args);
await contract.waitForDeployment();

```

2. 获取地址

```
const address = contract.target; // 同步
// 或
const address = await contract.getAddress(); // 异步
```

3. 获取部署交易

```
const tx = contract.deploymentTransaction;
if (tx) await tx.wait(confirmations);
```

4. 工具函数

```
ethers.parseEther("1.0") // 不再是 ethers.utils.parseEther
ethers.formatEther(balance) // 不再是 ethers.utils.formatEther
```

5. BigInt 运算

```
const result = num1 + num2; // 不再使用 .add()
const doubled = result * 2n; // 使用 BigInt 字面量
```

选择合适的部署方式

- 简单项目: 使用脚本部署 (方式一)
- 复杂项目: 使用 Hardhat Deploy 插件 (方式二)
- 开发测试: 在测试中部署 (方式三)

推荐工作流

1. 编写合约
2. 在测试中部署并测试
3. 使用脚本或 Hardhat Deploy 部署到测试网
4. 验证合约
5. 部署到主网

参考资源

- Ethers v6 官方文档: <https://docs.ethers.org/v6/>
- Hardhat 官方文档: <https://hardhat.org/>
- Hardhat Deploy 插件: <https://github.com/wighawag/hardhat-deploy>
- OpenZeppelin Upgrades: <https://docs.openzeppelin.com/upgrades-plugins/>
- Etherscan 验证: <https://docs.etherscan.io/tutorials/verifying-contracts-programmatically>